

Paralelização do ACO com CUDA e OpenMP para Seleção de Instâncias em Larga Escala

Raphael A. L. Soares¹, Wagner Victor A. Menezes¹,
Victor Gabriel R. Jácome¹, Daniel Rios Borges¹

¹Instituto de Informática – Universidade Federal de Goiás (UFG)
Caixa Postal 131 – CEP 74001-970 – Goiânia - GO – Brasil

{raphael.alves, wagnervictor, victor.ribeiro, daniel.rios}@discente.ufg.br

Abstract. *Instance selection reduces dataset size while preserving classifier quality, but Ant Colony Optimization (ACO), although effective, has an $O(N^2)$ cost that makes it infeasible at large scale: the sequential C++ baseline of Magalhães et al. (2024) does not scale to large bases. We parallelize ACO-based instance selection in OpenMP (CPU) and CUDA (GPU), running the 1-NN evaluation entirely on the GPU, and assess the impact of thread count, scheduler and block size. On nine datasets ($\leq 5k$ instances) a 24-core OpenMP run beats the GPU, but on a 253,680-instance base CUDA reaches $\sim 69\times$ over the sequential version (vs. $\sim 14\times$ for OpenMP), keeping the same quality ($\sim 23\%$ reduction, $F_1 \approx 0.77$). The GPU advantage is therefore scale-dependent: it dominates once the problem is large enough to amortize its overhead.*

Resumo. *A seleção de instâncias reduz o tamanho de bases de dados preservando a qualidade de classificadores, mas a Otimização por Colônia de Formigas (ACO), embora eficaz, tem custo $O(N^2)$ que a inviabiliza em larga escala: o baseline sequencial em C++ de Magalhães et al. (2024) não escala para grandes bases. Este trabalho paraleliza o ACO de seleção de instâncias em OpenMP (CPU) e CUDA (GPU), com a avaliação 1-NN inteiramente na GPU, e mede o impacto de threads, escalonadores e tamanho de bloco. Em nove bases ($\leq 5k$ instâncias) o OpenMP em 24 núcleos supera a GPU, mas em uma base de 253.680 instâncias o CUDA atinge $\sim 69\times$ sobre o sequencial (contra $\sim 14\times$ do OpenMP), preservando a qualidade ($\sim 23\%$ de redução, $F_1 \approx 0,77$). Conclui-se que o ganho da GPU é dependente de escala: ela domina quando o problema é grande o suficiente para amortizar seu custo fixo.*

1. Introdução

Modelos de aprendizado de máquina dependem de conjuntos de treinamento representativos, e bases reais frequentemente contêm instâncias redundantes ou ruidosas que aumentam o custo computacional sem agregar informação. A *seleção de instâncias* ataca esse problema: busca um subconjunto compacto dos dados que preserve (ou melhore) a acurácia de um classificador, reduzindo tempo e recursos de treinamento [Olvera-López et al. 2010].

A Otimização por Colônia de Formigas (ACO) [Dorigo et al. 2006] é uma metaheurística bioinspirada eficaz para seleção de instâncias quando usada

como método *wrapper*, guiando a escolha de instâncias por um classificador 1-NN [Cover and Hart 1967]. Sua principal limitação é o custo computacional: tanto o cálculo de distâncias quanto a avaliação 1-NN são $O(N^2)$ no número de instâncias N , o que torna o algoritmo caro à medida que a base cresce.

O baseline deste trabalho, Magalhães et al. [Magalhães et al. 2024], reescreveu o ACO de Python para C++ e obteve ganhos expressivos de desempenho mantendo a qualidade. Entretanto, essa versão permanece **sequencial**, e os próprios autores apontam que o alto custo computacional “limita sua escalabilidade, tornando-o inviável para grandes bases de dados”. Existe, portanto, uma lacuna clara: falta uma versão **paralela** do ACO de seleção de instâncias capaz de explorar CPUs multicore e GPUs para viabilizar o algoritmo em larga escala.

A paralelização do ACO é um tema bem estabelecido [Pedemonte et al. 2011], com implementações eficientes em CPU multicore via OpenMP [Abouelfarag et al. 2015] e em GPU via CUDA [Delévacq et al. 2013] que alcançam acelerações de uma a duas ordens de grandeza. Esses trabalhos, contudo, concentram-se em problemas de roteamento — sobretudo o caixeiro-viajante — nos quais o gargalo é a *construção de caminhos*. Na seleção de instâncias o gargalo é distinto: a avaliação 1-NN $O(N^2)$, uma carga *memory-bound* [Williams et al. 2009]. Não encontramos na literatura uma avaliação sistemática da paralelização (OpenMP e CUDA) do ACO de seleção de instâncias em larga escala.

Implementamos o ACO de seleção de instâncias em OpenMP (CPU) [Dagum and Menon 1998] e CUDA (GPU) [Nickolls et al. 2008], movendo a avaliação 1-NN inteiramente para a GPU, e avaliamos o impacto do número de threads, do escalonador de laço e do tamanho de bloco. Nas nove bases pequenas ($\leq 5k$ instâncias) o OpenMP em 24 núcleos supera a GPU, cujo custo fixo não se amortiza; já na base CDC Diabetes de 253.680 instâncias o CUDA atinge $\sim 69\times$ de aceleração sobre o sequencial (contra $\sim 14\times$ do OpenMP), preservando a qualidade da solução ($\sim 23\%$ de redução com $F_1 \approx 0,77$). O resultado central é que *o ganho da GPU é dependente de escala*.

2. Formulação do Problema

Seja um conjunto $X = \{x_1, \dots, x_N\}$ com N instâncias e F atributos. A seleção de instâncias procura um subconjunto $S \subseteq X$ que minimize $|S|$ mantendo a acurácia de um classificador treinado em S . No ACO, cada uma de K formigas constrói uma solução binária (seleciona ou não cada instância) guiada por dois fatores: o *feromônio* τ_i , que acumula a utilidade histórica de incluir a instância i , e a *visibilidade* η_i , derivada da distância média da instância às demais. A probabilidade de seleção é proporcional a $\tau_i^\alpha \eta_i^\beta$. A qualidade de cada solução é medida por um classificador 1-NN aplicado sobre o subconjunto selecionado, e o feromônio é reforçado nas soluções de maior F_1 e evaporado a cada iteração.

O custo computacional concentra-se em dois pontos $O(N^2F)$: o pré-cálculo de distâncias e, sobretudo, a avaliação 1-NN, na qual cada instância de teste busca seu vizinho mais próximo no subconjunto selecionado. Para $N = 253,680$, isso significa da ordem de 10^{10} comparações por iteração — o gargalo que motiva a paralelização.

3. Objetivos

1. Implementar o ACO de seleção de instâncias em três versões — sequencial (baseline), OpenMP (CPU) e CUDA (GPU) — com qualidade equivalente.
2. Superar o baseline sequencial em desempenho, preservando a qualidade da seleção (medida por F_1 e taxa de redução).
3. Avaliar a escalabilidade: em OpenMP, variando o número de threads e o escalonador de laço; em CUDA, variando o tamanho de bloco.
4. Reportar o *speedup* (absoluto e relativo) em bases de tamanhos distintos, incluindo uma base de larga escala (253.680 instâncias).

4. Metodologia

4.1. Algoritmo e implementações

As três versões compartilham o mesmo ACO de seleção guiada por feromônio ($\alpha = \beta = 1$, taxa de evaporação 0,1, $K = 64$ formigas). A construção de soluções é *embarçosamente paralela*: cada formiga escreve apenas em sua fatia da colônia e lê o vetor de probabilidades como dado somente-leitura, com gerador aleatório semeado por (*iteração*, k) para garantir determinismo independente do paralelismo.

Na versão **OpenMP**, paralelizam-se a construção das formigas, o depósito de feromônio (com `atomic` para evitar condições de corrida) e, principalmente, a avaliação 1-NN — o laço quente, que responde por mais de 98% do tempo. Esse laço usa `schedule(runtime)`, permitindo controlar o escalonador via a variável `OMP_SCHEDULE`. Na versão **CUDA**, a avaliação 1-NN roda inteiramente na GPU (`knn_1nn_kernel`: uma thread por instância de teste), eliminando o transporte da colônia $K \times N$ a cada iteração; para N grande, o feromônio é mantido como vetor 1D e as distâncias são calculadas *on-the-fly*, evitando a matriz $N \times N$.

4.2. Configuração experimental

Todos os experimentos rodaram na **mesma máquina**: Intel Core i9-14900K (24 núcleos físicos — 8 *performance* + 16 *efficiency* — e 32 threads) e GPU NVIDIA GeForce RTX 4090 (24 GB). Avaliamos nove bases do baseline (de 299 a 4.981 instâncias) e a base **CDC Diabetes Health Indicators** [Centers for Disease Control and Prevention 2015] (253.680 instâncias, 21 atributos). Para a comparação dos nove datasets, fixamos 100 iterações em todos os modos, garantindo comparação justa de tempo. No experimento de larga escala (CDC) habilitamos *early stopping* (parada após 20 iterações sem melhora). As métricas são F_1 , taxa de redução, tempo de execução, vazão (GFLOPS) e *speedup*. Para o CDC, o *speedup* é normalizado por iteração, usando como referência (T_1) uma execução de 1 thread da mesma campanha. O código e os dados estão disponíveis publicamente¹.

4.3. Métricas

A qualidade da seleção é avaliada por um classificador 1-NN treinado sobre o subconjunto selecionado S , por meio de quatro métricas: *acurácia* (fração de classificações corretas), *precisão* (fração de positivos preditos que estão corretos), *recall* (fração de positivos reais

¹<https://github.com/phael-exe/aco-selection-parallel>

recuperados) e F_1 , a média harmônica de precisão e *recall* ($F_1 = 2 \frac{\text{precisão} \cdot \text{recall}}{\text{precisão} + \text{recall}}$). A compactação é medida pela *taxa de redução* $1 - |S|/N$ (maior é melhor). O desempenho é quantificado pelo tempo de execução do ACO, pela vazão da avaliação 1-NN em GFLOPS (operações de ponto flutuante por segundo) e pelo *speedup*, a razão entre o tempo de uma referência e o tempo da configuração avaliada.

5. Resultados

5.1. Desempenho em bases de pequena e média escala

A Tabela 1 reporta a qualidade da seleção nas nove bases: as três implementações convergem para soluções equivalentes (variações de F_1 dentro de $\sim 2\%$, compatíveis com o ruído do algoritmo estocástico), removendo de 6% a 22% das instâncias. Quanto ao tempo (Figura 1 e Tabela 2), o *speedup* da CUDA cresce com N — de $0,7\times$ na menor base a $8,2\times$ na maior —, mas em **todas** as nove bases o OpenMP em 24 núcleos é mais rápido que a GPU: o custo fixo de lançamento de kernels e de transferência host-device não se amortiza enquanto o problema é pequeno. Em bases de poucos milhares de instâncias, a CPU multicore é a melhor escolha.

Tabela 1. Qualidade da seleção nas nove bases (classificador 1-NN, configuração sequencial; as três implementações convergem para qualidade equivalente).

Dataset	N	Acur.	Prec.	Recall	F_1	Red. (%)
heart_failure	299	0,926	0,878	0,896	0,887	19,4
haberman	306	0,935	0,851	0,914	0,881	19,9
cirrhosis	418	0,880	0,893	0,849	0,867	21,5
diabetes	768	0,935	0,916	0,896	0,906	19,5
tic-tac-toe	958	0,985	0,991	0,967	0,979	6,7
yeast	1484	0,978	0,974	0,984	0,979	6,3
vaccine	3152	0,980	0,961	0,955	0,958	12,7
Employee	4653	0,867	0,798	0,821	0,809	22,2
brain-stroke	4981	0,972	0,705	0,750	0,727	22,4

Tabela 2. *Speedup* sobre o sequencial nas nove bases (100 iterações). A melhor configuração OpenMP é a mais rápida entre threads e escalonadores.

Dataset	N	Seq. (ms)	CUDA	OpenMP (melhor)
heart_failure	299	82,1	$0,7\times$	$2,9\times$
haberman	306	72,2	$1,8\times$	$3,6\times$
cirrhosis	418	161,4	$0,9\times$	$4,0\times$
diabetes	768	275,8	$1,5\times$	$4,5\times$
tic-tac-toe	958	670,6	$1,8\times$	$5,8\times$
yeast	1484	861,2	$2,4\times$	$6,5\times$
vaccine	3152	16439,7	$5,9\times$	$13,0\times$
Employee	4653	5442,5	$7,4\times$	$10,4\times$
brain-stroke	4981	7773,9	$8,2\times$	$11,5\times$

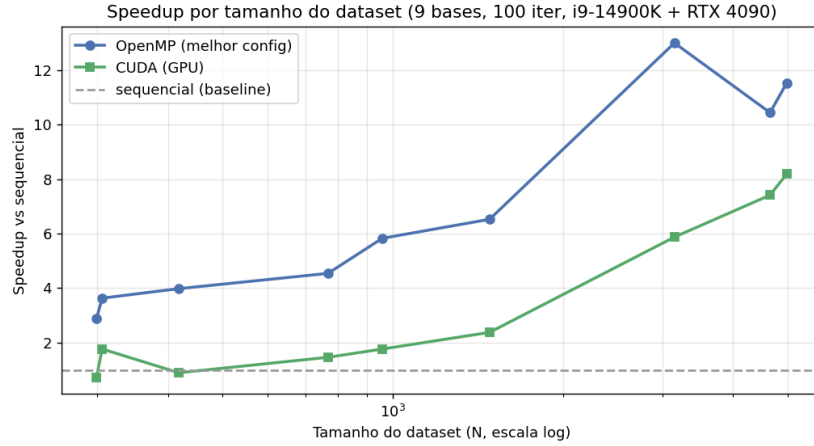


Figura 1. O ganho da GPU cresce com o tamanho da base, mas até ~5k instâncias o OpenMP em 24 núcleos supera a CUDA: o custo fixo da GPU só se amortiza em escala maior.

5.2. Efeito do escalonador OpenMP

A avaliação 1-NN é uma carga *irregular* (instâncias com vizinhos próximos terminam mais cedo), o que favorece escalonadores que rebalanceiam a carga. A Figura 2 compara, com 16 threads, os escalonadores *static* (partição em blocos) e *dynamic* (distribuição sob demanda) e seus tamanhos de *chunk*. Nas bases maiores, o *dynamic* é em média ~25–30% mais rápido que o *static*, enquanto o tamanho de *chunk* (1, 2 ou 3) tem efeito secundário. O balanceamento dinâmico compensa porque cada iteração do laço quente é pesada o suficiente para diluir o custo da fila compartilhada. O resultado está alinhado a estudos anteriores de *tuning* de escalonadores e de número de threads em ACO paralelo [Abouelfarag et al. 2015].

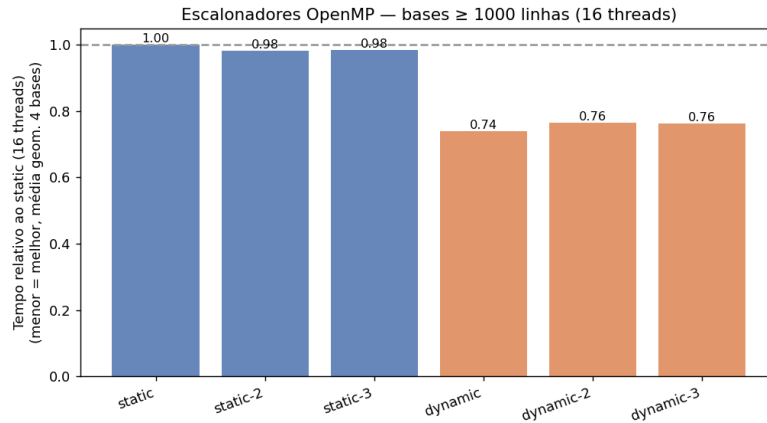


Figura 2. Tempo relativo ao *static* (16 threads, média geométrica das bases ≥1000 instâncias): o *dynamic* reduz o tempo em ~25–30%; o tamanho de *chunk* é secundário.

5.3. Desempenho na base de larga escala

Na base CDC o quadro se inverte. O OpenMP escala bem com o número de threads (Tabela 3 e Figura 3), chegando a ~14× (*static*) e ~16× (*dynamic*) sobre 1 thread

ao usar as 32 threads do i9-14900K; a vantagem do `dynamic` concentra-se nos maiores graus de paralelismo. A CUDA (Tabela 4) atinge **54–69×** sobre o sequencial, executando a campanha completa em ~ 2 minutos — a melhor configuração CUDA é $\sim 4,3\times$ mais rápida que a melhor configuração OpenMP.

Tabela 3. OpenMP na base CDC por número de threads (*early stopping* em 22 iterações; *speedup* normalizado por iteração vs. 1 thread; *sp.* = *speedup*).

Threads	static (min)	<i>sp.</i>	dynamic (min)	<i>sp.</i>
2	109,2	1,25×	118,7	1,15×
4	60,2	2,26×	57,9	2,35×
8	33,0	4,13×	36,2	3,77×
16	16,5	8,27×	18,1	7,53×
32	9,7	14,01×	8,6	15,91×

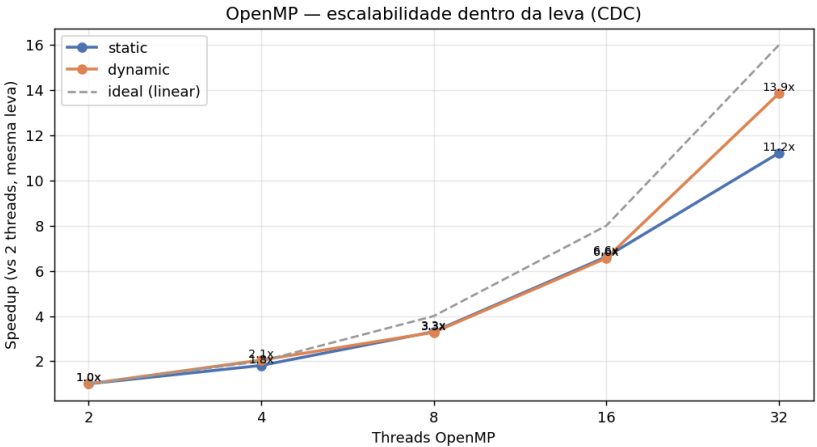


Figura 3. Escalabilidade do OpenMP na base CDC: o *speedup* relativo a 2 threads cresce de forma quase linear até 32 threads, com leve vantagem do `dynamic` no topo.

A varredura de tamanho de bloco da CUDA (Figura 4) revela uma curva em U: os extremos (bloco 32 e 1024) são mais rápidos e o meio (64–128) mais lento, com diferença de $\sim 28\%$ entre o melhor e o pior. A explicação está no padrão de acesso do `knn_lnn_kernel`: todas as threads leem a mesma linha de referência simultaneamente, em *broadcast*. Blocos grandes maximizam o reúso desse *broadcast*, favorecidos pela coalescência de memória [NVIDIA Corporation 2024]; blocos pequenos, embora de menor ocupância, mantêm o desempenho — pois ocupância alta não implica melhor desempenho quando há paralelismo de instruções e de memória suficiente [Volkov 2010]; o meio perde nos dois efeitos. O padrão é reproduzível (variação $< 0,2\%$ entre execuções repetidas), confirmando que não é ruído.

Em todos os modos a solução converge para a mesma qualidade: $\sim 23\%$ de redução de instâncias mantendo $F_1 \approx 0,77$ — a paralelização altera o tempo, não o resultado. O *early stopping* reduziu cada execução para 22 iterações, economizando $\sim 78\%$ do tempo sem perda de qualidade.

Tabela 4. CUDA na base CDC por tamanho de bloco (tempo, vazão e *speedup* normalizado por iteração vs. 1 thread). A qualidade é idêntica em todos os blocos.

Block size	Tempo (s)	GFLOPS	Speedup
32	119,7	573,0	68,4×
64	151,4	452,9	54,0×
128	149,9	457,3	54,6×
256	135,2	507,0	60,5×
512	127,9	536,0	63,9×
1024	118,5	578,9	69,1×

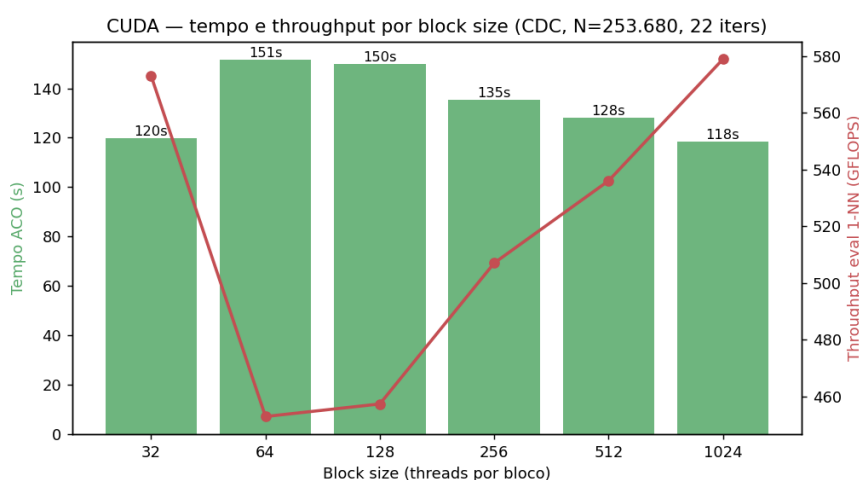


Figura 4. CUDA na base CDC: a curva de tempo e vazão por tamanho de bloco tem forma de U — os extremos vencem por equilibrarem reuso de *broadcast* e ocupância dos SMs.

6. Análise Crítica

Os resultados sustentam a tese de escalabilidade, mas alguns pontos merecem leitura cuidadosa.

O ponto de cruzamento depende do hardware. A constatação de que o OpenMP supera a GPU em bases pequenas é contingente à CPU empregada: o i9-14900K, com 24 núcleos, é particularmente robusto. Em uma CPU mais modesta, o ponto de cruzamento se deslocaria para bases menores e a GPU venceria antes. O resultado deve ser lido como “a GPU compensa acima de um limiar de tamanho”, e não como superioridade absoluta de um paradigma.

A explicação da curva em U é fundamentada, mas não medida. A atribuição do efeito ao equilíbrio entre reuso de *broadcast* e ocupância é coerente com a literatura [Volkov 2010, NVIDIA Corporation 2024], mas não foi confirmada por *profiling* do nosso kernel (ocupância alcançada, vazão de memória e taxa de acerto de cache via Nsight Compute). A reprodutibilidade (<0,2%) garante que o padrão é real, não que a causa proposta seja a única.

O baseline de *speedup* do CDC é uma escolha metodológica. Como executar

o sequencial completo em 253 mil instâncias levaria horas, o tempo de referência (T_1) é uma execução de 1 thread da mesma campanha, com o *speedup* normalizado por iteração — justo (mesmo código e máquina), mas cuja magnitude absoluta depende dessa normalização. Valores de *speedup* entre trabalhos só são comparáveis sob a mesma metodologia.

Heterogeneidade de núcleos. O i9-14900K combina 8 núcleos de desempenho (com *hyperthreading*) e 16 de eficiência; a escalabilidade de 16 para 32 threads mistura esses tipos de núcleo, tornando a curva de *speedup* menos regular que a de uma CPU homogênea.

Generalização e comparação com a literatura. Avaliamos uma única base de larga escala, e a taxa de redução variou de 6% a 22% entre os datasets — o benefício da seleção é dependente dos dados. Delévacq et al. [Delévacq et al. 2013] reportam $\sim 23\times$ para o ACO em GPU no caixeiro-viajante; nosso $\sim 69\times$ refere-se a outro problema, outra carga (avaliação 1-NN *memory-bound*) e outro hardware, não sendo diretamente comparável, mas situando-se na faixa de uma a duas ordens de grandeza típica de implementações de ACO em GPU [Pedemonte et al. 2011].

7. Conclusão

Este trabalho paralelizou o ACO de seleção de instâncias em OpenMP e CUDA, preenchendo a lacuna de escalabilidade do baseline sequencial de Magalhães et al. [Magalhães et al. 2024] e viabilizando o algoritmo em uma base de 253.680 instâncias. O resultado central é que **o ganho da GPU é dependente de escala**: em bases de até $\sim 5k$ instâncias o OpenMP em 24 núcleos é mais rápido, enquanto em larga escala a CUDA domina, atingindo $\sim 69\times$ sobre o sequencial e sendo $\sim 4,3\times$ mais rápida que a melhor configuração de CPU — sempre preservando a qualidade da seleção ($\sim 23\%$ de redução, $F_1 \approx 0,77$).

Como trabalhos futuros, destacam-se: confirmar a explicação da curva em U por *profiling* (Nsight Compute); investigar a *co-execução híbrida* CPU+GPU, particionando o trabalho entre os dispositivos; e estender a avaliação a mais bases de larga escala, aproximando o algoritmo de aplicações reais de mineração de dados.

Referências

- Abouelfarag, A. A., Mohamed Aly, W., and Elbialy, A. G. (2015). Performance analysis and tuning for parallelization of ant colony optimization by using OpenMP. In *Computer Information Systems and Industrial Management (CISIM)*, volume 9339 of *LNCS*, pages 73–85. Springer.
- Centers for Disease Control and Prevention (2015). CDC diabetes health indicators dataset (BRFSS 2015). UCI Machine Learning Repository. 253.680 instâncias, 21 atributos.
- Cover, T. M. and Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.
- Dagum, L. and Menon, R. (1998). OpenMP: An industry standard API for shared-memory programming. *IEEE Computational Science and Engineering*, 5(1):46–55.

- Delévacq, A., Delisle, P., Gravel, M., and Krajecki, M. (2013). Parallel ant colony optimization on graphics processing units. *Journal of Parallel and Distributed Computing*, 73(1):52–61.
- Dorigo, M., Birattari, M., and Stützle, T. (2006). Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4):28–39.
- Magalhães, J. M. d. O., Gonçalves, A. C. M., Nobre, C. N., and Freitas, H. C. d. (2024). Avaliação de desempenho e escalabilidade do algoritmo de otimização de colônia de formigas em C++ e Python. Código disponível em: <https://github.com/jmarcosjm/aco-cpp>.
- Nickolls, J., Buck, I., Garland, M., and Skadron, K. (2008). Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53.
- NVIDIA Corporation (2024). CUDA C++ best practices guide. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- Olvera-López, J. A., Carrasco-Ochoa, J. A., Martínez-Trinidad, J. F., and Kittler, J. (2010). A review of instance selection methods. *Artificial Intelligence Review*, 34(2):133–143.
- Pedemonte, M., Nesmachnow, S., and Cancela, H. (2011). A survey on parallel ant colony optimization. *Applied Soft Computing*, 11(8):5181–5197.
- Volkov, V. (2010). Better performance at lower occupancy. In *GPU Technology Conference (GTC)*. NVIDIA. University of California, Berkeley.
- Williams, S., Waterman, A., and Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.